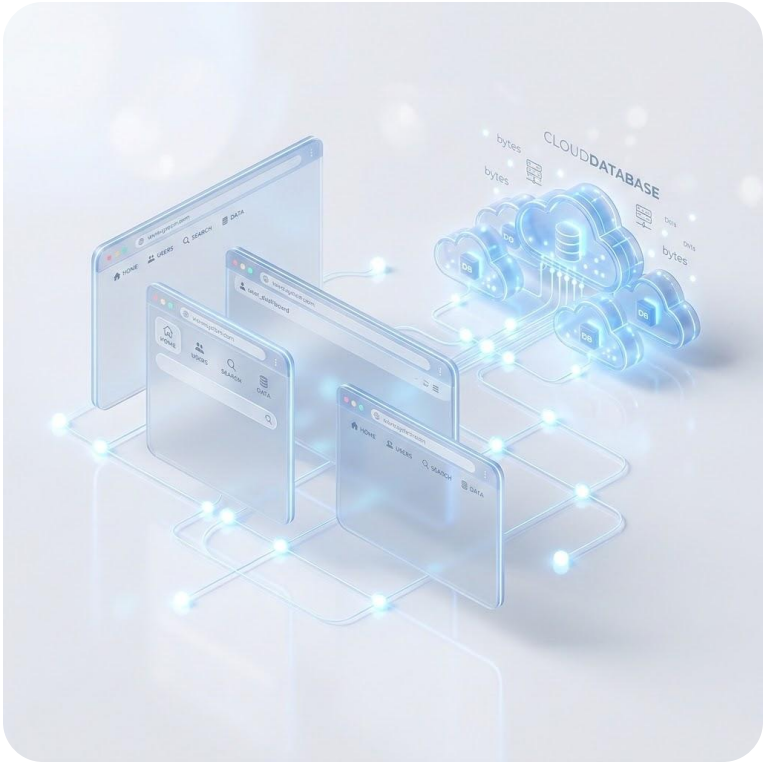
Departamento de Informática
UFV Universidade Federal de Viçosa

INF 321

Projeto e Desenvolvimento de Sistemas para a Web

Unidade 2.2:
Desenvolvimento Front-End |> CSS

Prof. Lucas Vegi



Cascading Style Sheets

Transformando estrutura em apresentação visual

HTML

estrutura e significado

CSS

aparência e apresentação

Onde estamos?

Na aula anterior



HTML

Estrutura da página

Agora



CSS

Apresentação da página

E posteriormente



JavaScript

Comportamento e interação

HTML × CSS

HTML estrutura. CSS apresenta.

Considere o Código

```
<h1>Produtos</h1>
<p>Confira nossos...</p>
<button>Comprar</button>
```

O HTML define

Estrutura

- Título (<h1>)
- Parágrafo (<p>)
- Botão (<button>)

O CSS pode definir

Apresentação

- Cor e Fonte
- Tamanho e Posição
- Espaçamento
- Bordas e Fundo

O que é CSS?



Significado

Cascading Style Sheets

É uma linguagem padrão utilizada principalmente para definir a **apresentação visual** de documentos estruturados em HTML.



Com CSS podemos controlar:

- cores
- fontes
- tamanhos
- espaçamentos
- bordas
- dimensões
- posicionamento
- layouts
- responsividade
- animações

Primeiro exemplo

HTML



Estrutura

Define o elemento e o conteúdo textual da página.

```
<h1>Minha página</h1>
```

CSS



Estilização

Aplica regras visuais aos elementos selecionados.

```
h1 {
  color: blue;
}
```

RESULTADO



Apresentação

Como o navegador exibe o elemento estruturado e estilizado.

Minha página

Anatomia de uma regra CSS

EXEMPLO PRÁTICO

```
h1 {
  color: blue;
}
```

h1 Seletor (seleciona o elemento)

color Propriedade (o que será alterado)

blue Valor (a nova definição)

ESTRUTURA GERAL

```
seletor {
  propriedade: valor;
}
```

Seletor Aponta para o elemento HTML

Propriedade Atributo que deseja estilizar

Valor Configuração aplicada à propriedade

Importante: Uma regra pode ter múltiplos seletores separados por vírgula (,). Ex: `h1, h2, p { color: blue;}`

Várias propriedades

Código Exemplo

Regra com Múltiplas Propriedades

```
h1 {
    color: blue;
    font-size: 32px;
    text-align: center;
}
```

Regra de Sintaxe

Declarações no CSS

Podemos agrupar diversas propriedades de estilo para um mesmo seletor dentro das chaves.

- Cada par de **propriedade: valor** constitui uma declaração individual.

Importante: Cada declaração deve obrigatoriamente terminar com ponto e vírgula (`;`).

Comentários em CSS

CÓDIGO EXEMPLO

Sintaxe de Comentário

```
/* Este é um comentário de uma linha */
h1 {
  /* Estilo do título principal */
  color: blue;
  font-size: 32px;
}
```

BOAS PRÁTICAS

Uso de Comentários

- **Sintaxe**: São delimitados por `/*` para abrir e `*/` para fechar as anotações.
- **Organização**: Assim como no HTML, ajudam a documentar a estrutura das folhas de estilo.

Atenção: Não devem conter informações sensíveis, pois são baixados e lidos pelo navegador.

CSS inline

Como funciona



Uso Direto no Elemento

Podemos escrever CSS diretamente no elemento HTML através do atributo style. É o chamado **CSS inline**.

```
<h1 style="color: blue;">Produtos</h1>
```

O Problema

Dificuldade de Manutenção

Se tivermos muitos elementos na página, a estilização precisa ser replicada individualmente em cada um deles.

```
<h1 style="color: blue;">
<h2 style="color: blue;">
<p style="color: blue;">
```

Atenção: Multiplicar regras inline torna o código HTML poluído e inviabiliza alterações globais rápidas.

CSS interno

CÓDIGO EXEMPLO



Uso dentro de <style>

```
<head>
  <style>
    h1 { color: blue; }
    p { font-size: 18px; }
  </style>
</head>
```

CARACTERÍSTICAS

Estilização na Página

- **Localização:** Escrito dentro do cabeçalho `<head>` da página HTML utilizando a tag `<style>`.
- **Escopo:** As regras se aplicam a todos os elementos correspondentes daquela página específica.

Recomendação: Funciona muito bem para exemplos práticos, protótipos e *Single Page Application* (SPA).

CSS externo

CÓDIGO EXEMPLO



Abordagem mais comum

O estilo é definido em arquivo **style.css** separado e vinculado ao **index.html**:

```
<head>
  <link rel="stylesheet" href="style.css">
</head>
```

```
/* Conteúdo de style.css */
h1 { color: blue; }
```

VANTAGENS

Manutenibilidade e Organização

- **Separação de Conceitos:** O CSS fica totalmente isolado do código estrutural HTML.
- **Reutilização de Código:** Um único arquivo CSS pode estilizar todas as páginas de um portal web de forma global.

Recomendação: Esta é a abordagem padrão de mercado, otimizando o carregamento da página por meio do cache do navegador.

Seletor por `class`

CÓDIGO EXEMPLO

Sintaxe HTML e CSS

```
<!-- HTML -->
<p class="destaque">Produto...</p>

/* CSS */
.destaque { color: red; }
```

CONCEITO

Identificação por Classe

- **Observe o Ponto (.)**

No CSS, a classe é sempre precedida por um ponto (ex: `.destaque`).

- **Vinculação**

No HTML, usamos o atributo `class="nome"` (sem o ponto).

Dica: Diferente dos IDs, uma mesma classe pode ser reutilizada em múltiplos elementos HTML na mesma página.

Reutilizando classes

CÓDIGO EXEMPLO



Uso em Múltiplos Elementos

Uma mesma classe CSS pode ser aplicada livremente a várias tags HTML simultaneamente.

```
<!-- HTML -->
<p class="destaque">Produto em promoção!</p>
<p class="destaque">Últimas unidades!</p>

/* CSS */
.destaque { color: red; }
```

CONCEITO

Estilização Centralizada

- **Reutilização Máxima:**
  Evita a repetição de código CSS inline. Você define a regra visual apenas uma vez e a distribui por todo o documento HTML.
- **Manutenção Facilitada:**
  Caso precise alterar a cor ou fonte no futuro, basta modificar uma única linha na regra da classe para atualizar todos os elementos vinculados.

Resultado: Todos os parágrafos marcados com a classe receberão a cor vermelha de forma automática e padronizada.

Seletor por `id`

CÓDIGO EXEMPLO

Sintaxe HTML e CSS

```
<!-- HTML -->
<h1 id="titulo">Produtos</h1>

/* CSS */
#titulo { color: blue; }
```

CONCEITO

Identificação por ID

- **Observe o Símbolo (#)**
  No CSS, o seletor de ID é sempre precedido por uma cerquilha (ex: `#titulo`).

- **Vinculação**
  No HTML, vinculamos usando o atributo `id="titulo"` (sem a cerquilha).

Importante: Diferente das classes, um ID deve ser único e exclusivo para um único elemento na página HTML.

Classe × ID

ATRIBUTO

Class (Classe)

Pode ser **reutilizada** em múltiplos elementos HTML na mesma página.

```
<!-- HTML -->
<p class="destaque">Promoção!</p>
<p class="destaque">Frete grátis!</p>
```

ATRIBUTO

ID (Identificador)

Identifica um **elemento específico** e exclusivo dentro do seu documento HTML.

```
<!-- HTML -->
<h1 id="titulo">Produtos</h1>
```

Resumo Prático: Use classes sempre que precisar reutilizar estilos visuais, e IDs apenas para elementos de destaque único ou manipulação de scripts.

Seletores combinados

CÓDIGO EXEMPLO

Sintaxe HTML e CSS



```
<!-- HTML -->
<p class="destaque">Aplica-se</p>
<div class="destaque">Não se aplica</div>
/* CSS */
p.destaque { color: red; }
```

CONCEITO

Combinação de Elemento e Classe

- **Restrição Direta**
  A regra `p.destaque` filtra o elemento e a classe juntos, garantindo que o estilo não "vaze" para outras tags.

- **Alta Precisão**
  Permite reutilizar a classe `.destaque` em `divs` ou `spans` sem que herdem o estilo específico do parágrafo.

Dica: A combinação de seletores aumenta a especificidade, ajudando a organizar melhor regras estruturais sem criar novas classes.

Seletores descendentes

CÓDIGO EXEMPLO

Sintaxe HTML e CSS



```
<!-- HTML -->
<div class="card"> ... <p>R$ 5.000</p> </div>
/* CSS */
.card p { color: gray; }
```

CONCEITO

Seleção por Hierarquia

- **Relação de Descendência**
  Seleciona elementos específicos que estão contidos dentro de outros elementos (independentemente do nível de aninhamento).

- **Uso do Espaço**
  No CSS, um caractere de espaço em branco entre seletores (ex: `.card p`) representa essa relação.

Resultado: O CSS selecionará e aplicará a cor cinza exclusivamente para a tag `<p>` que estiver dentro de um container com a classe `.card`.

Formatos de Cores no CSS

CÓDIGO EXEMPLO

Sintaxe de Aplicação

```css
h1 { color: blue; } /* Nome */
h1 { color: #2563eb; } /* Hex */
h1 { color: rgb(37, 99, 235); }
```

CONCEITO

Três Formas de Definição

- **Nomes de Cores**
  Palavras-chave diretas como `blue` ou `red`.
- **Hexadecimal (#RRGGBB)**
  Representação de alta precisão (ex: `#2563eb`).
- **Função RGB**
  Combinação de canais Vermelho, Verde e Azul.

Dica Prática: Hexadecimal e RGB oferecem mais de 16 milhões de cores, enquanto a lista de nomes predefinidos possui apenas 140 cores.

Cor do texto × fundo

CÓDIGO EXEMPLO

Aplicação de Cores

```
body {
  color: #222; /* texto */
  background-color: #f5f5f5; /* fundo */
}
```

CONCEITO

Propriedades de Cor

`color`
Define a cor do conteúdo de primeiro plano (neste caso, o `texto` do documento).

`background-color`
Define a cor do fundo do elemento (neste caso, o `fundo` da página).

Dica Prática: Utilize um bom contraste entre a cor do texto e a cor de fundo para garantir a acessibilidade e legibilidade do seu conteúdo da Web.

Cor do texto × fundo (contraste para daltônicos)





Tamanho da fonte

CÓDIGO EXEMPLO

Definição no CSS

```css
h1 {
  font-size: 32px;
}
p {
  font-size: 16px;
}
```

CONCEITO

Unidades de Medida

- **Absolutas vs. Relativas**
  O CSS suporta diferentes tipos de unidades para dimensionar textos de forma fixa ou flexível: `px`, `rem`, `em`, e `%`.
- **Flexibilidade e Escala**
  O uso de unidades relativas como `rem` facilita a acessibilidade, adaptando o texto ao tamanho padrão do navegador.

Foco do Aprendizado: Neste primeiro momento, vamos trabalhar principalmente com as unidades `px` e `rem` para construir nossos layouts.

Unidades de medidas

px (Pixel)

É uma unidade absoluta e fixa. Um elemento com `16px` terá sempre esse tamanho exato, sem se adaptar às preferências do usuário.

rem (Root Em)

Relativa ao tamanho da fonte raiz (`<html>`, padrão `16px`). O `rem` se ajusta sozinho para melhor acessibilidade.

em

Relativa ao elemento pai mais próximo. Se o pai tem `20px` e o filho `2em`, o filho terá `40px`. Pode acumular em cascata.

% (Porcentagem)

Relativa ao valor da mesma propriedade no pai. Funciona muito bem para larguras (`width: 50%`), alturas e tamanhos de layout.

Família da fonte

CÓDIGO EXEMPLO

Sintaxe de Fontes no CSS

```css
/* Fonte única + alternativa genérica */
body { font-family: Arial, sans-serif; }

/* Sequência de fontes (Fallback) */
body { font-family: "Helvetica Neue", Arial,
sans-serif; }
```

CONCEITO

Sequência de Fontes

- **Fallback System**
  Permite definir uma lista ordenada. Se a primeira fonte não estiver disponível no sistema do usuário, o navegador tentará as próximas de forma sequencial.

- **Família Genérica Obrigatória**
  Sempre termine a lista com uma família genérica (ex: `sans-serif`) para garantir que o navegador use um tipo de fonte apropriado.

Como funciona: Se a primeira fonte não estiver disponível no computador ou dispositivo de quem acessa, o navegador tenta aplicar a próxima listada.

Peso e estilo da fonte

CÓDIGO EXEMPLO

Sintaxe de Aplicação

```
h1 { font-weight: bold; }
h1 { font-weight: 700; } /* Numérico */

p {
  font-style: italic;
}
```

CONCEITO

Controle Visual do Texto

- **font-weight**
  Controla a espessura do texto. Aceita valores descritivos (ex: `bold`) ou numéricos (ex: `700`).

- **font-style**
  Controla a inclinação do texto, permitindo aplicar o estilo `italic` para ênfase visual.

Dica Prática: O valor numérico `700` equivale diretamente ao negrito (`bold`), enquanto `400` representa o peso padrão (`normal`).

Alinhamento

CÓDIGO EXEMPLO

Sintaxe de Alinhamento

```css
h1 {
  text-align: center;
}
p {
  text-align: justify;
}
```

CONCEITO

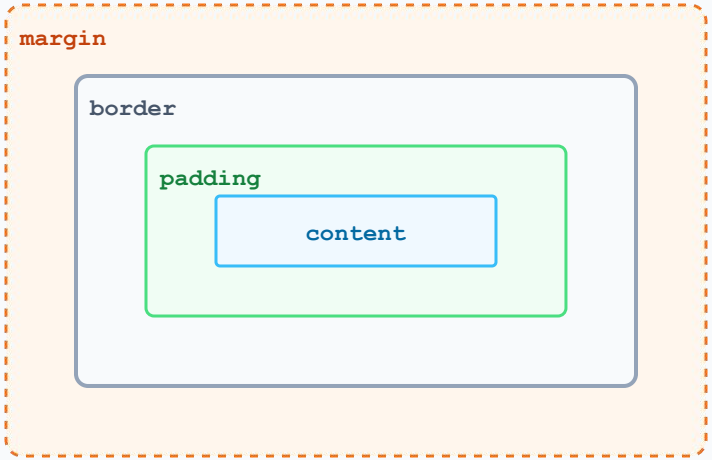
Propriedade `text-align`

- **Controle de Alinhamento**
  Define o alinhamento horizontal do texto inline dentro do seu elemento bloco pai.

- **Valores Comuns**
  O CSS oferece opções clássicas para posicionamento: `left`, `center`, `right`, e `justify`.

Dica Prática: O alinhamento justificado (`justify`) distribui o texto uniformemente, mas deve ser usado com moderação na Web para evitar grandes espaços vazios entre as palavras.

O Box Model



CONCEITO

Anatomia do Elemento



- **margin:**
Limpa uma área externa ao redor da borda. É completamente transparente.

- **border:**
Uma borda que envolve o preenchimento (padding) e o conteúdo.

- **padding:**
Limpa uma área ao redor do conteúdo. Também é transparente*.

- **content:**
O conteúdo real da caixa, como texto, imagens ou outros elementos.

Regra de Ouro: Todo elemento HTML pode ser pensado como uma caixa, e o Box Model define como essas propriedades interagem para calcular seu tamanho final.

padding

CÓDIGO EXEMPLO

Sintaxe de Aplicação

```css
/* Define o espaçamento interno */
.card {
  padding: 20px;
}
```

CONCEITO

O Espaço Interno

- **Definição de Preenchimento**
  O padding limpa uma área ao redor do conteúdo, definindo a distância interna entre o conteúdo do elemento e a sua borda delimitadora.

- **Comportamento do Fundo**
  Diferente da margem externa, a cor de fundo ou imagem definida no elemento se estende por toda a área ocupada pelo padding.

Como funciona: O `padding` aumenta as dimensões totais da caixa do elemento na tela, a não ser que a propriedade `box-sizing: border-box` seja configurada.

margin

CÓDIGO EXEMPLO

Sintaxe de Aplicação

```css
/* Define o espaçamento externo */
.card {
  margin: 20px;
}
```

CONCEITO

O Espaço Externo

- **Definição de Margem**
  A margem limpa uma área ao redor do elemento, definindo a distância externa de segurança entre a sua borda e os elementos vizinhos.

- **Transparência Total**
  A margem é completamente transparente, não possui cor de fundo própria e sempre revela o fundo do elemento pai.

Dica Prática: Diferente do `padding`, a margem não herda a cor de fundo do elemento, servindo estritamente para empurrar e espaçar elementos adjacentes.

border

CÓDIGO EXEMPLO

Sintaxe de Aplicação

```css
/* Define a borda do elemento */
.card {
  border: 1px solid #e3e3e3;
}
```

CONCEITO

O Espaço Delimitador

- **Definição de Borda**

  A border envolve o preenchimento (padding) e o conteúdo, servindo como um limite visual personalizável ao redor da caixa do elemento.

- **Sintaxe e Estilização**

  Pode ser customizada em largura (width), estilo (solid, dashed, dotted) e cor para destacar e estruturar os componentes.

Dica Prática: A propriedade border pode ser dividida em `border-width`, `border-style` e `border-color` para um controle mais específico de cada lado da caixa.

width e height

CÓDIGO EXEMPLO

Sintaxe de Aplicação

```css
/* Controle de Dimensões */
.card {
  width: 300px; /* Fixo */
  height: 200px;
}
.banner {
  width: 50%; /* Relativo */
}
```

CONCEITO

Dimensionamento de Caixa

- **Unidades Absolutas (px)**
  O pixel (px) define um tamanho rígido e físico na tela. Útil para elementos que não devem mudar de proporção, como ícones ou botões fixos.

- **Unidades Relativas (%)**
  A porcentagem (%) calcula o tamanho baseando-se no elemento pai. Essencial para criar designs fluidos e responsivos.

Dica de Layout: Por padrão, as propriedades de largura e altura controlam apenas a área do conteúdo interno. Use `box-sizing: border-box` para incluir bordas e preenchimentos neste cálculo.

display

CÓDIGO EXEMPLO

Sintaxe de Aplicação

```css
/* Altera o comportamento */
div {
  display: block;
}
```

CONCEITO

Comportamento de Fluxo

- **Blocos e Linhas**

  Elementos `block` ocupam toda a largura disponível. `inline` fluem com o texto. `inline-block` aceita largura/altura sem quebrar linha.

- **Flexbox e Grid**

  Valores `flex` e `grid` ativam modelos modernos de diagramação unidimensional ou bidimensional para seus filhos diretos.

Dica Prática: O valor `none` oculta totalmente o elemento e o remove do fluxo de renderização, liberando seu espaço na tela.

block × inline

COMPORTAMENTO

Elementos em Bloco (Block)

Ocupa toda a largura disponível da linha pai. Inicia sempre em uma nova linha do fluxo da página.

```
<div>Primeiro</div>
<div>Segundo</div>

/* Resultado Visual */
Primeiro
Segundo
```

COMPORTAMENTO



Elementos em Linha (Inline)

Ocupa apenas a largura necessária para o seu próprio conteúdo, mantendo-se na mesma linha.

```
<span>Primeiro</span>
<span>Segundo</span>

/* Resultado Visual */
PrimeiroSegundo
```

Dica Prática: Elementos do tipo `block` aceitam configurações de largura e altura, enquanto elementos `inline` ignoram essas propriedades por padrão.

inline-block

CÓDIGO EXEMPLO

Sintaxe de Aplicação

```css
/* Ativa o comportamento */
.item {
  display: inline-block;
  width: 120px;
  height: 60px;
}
```

CONCEITO



Visualização do Fluxo

Os elementos ficam **na mesma linha**, mas respeitam as configurações de `width` e `height`.

Item 1 Item 2 Item 3

Dica Prática: Útil para elementos que precisam ficar lado a lado, mas ainda devem se comportar como caixas (com largura, altura e paddings controláveis).

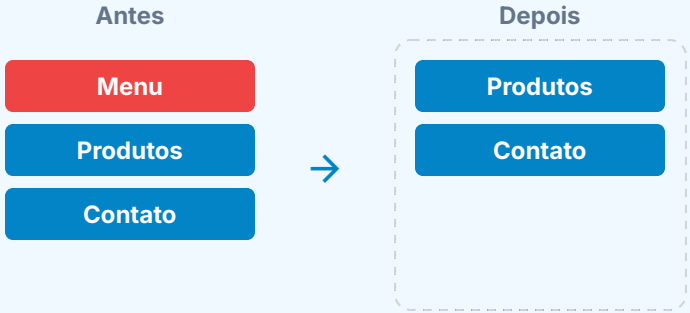
none

CÓDIGO EXEMPLO

Sintaxe de Aplicação

```css
/* Remove do layout */
.menu {
  display: none;
}
```





Dica Prática: É diferente de usar `visibility: hidden`. Com `display: none`, o elemento deixa totalmente de existir para o cálculo de layout da página.

grid

CÓDIGO EXEMPLO

Sintaxe de Aplicação

```css
/* Transforma em CSS Grid */
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}
```

CONCEITO



Visualização do Fluxo

Transforma o elemento em um **container CSS Grid**, distribuindo os itens em linhas e colunas.

Item 1	Item 2	Item 3
Item 4	Item 5	Item 6

Dica Prática: O `Flexbox` (ver próximo slide) é ideal para organizar elementos em uma dimensão, enquanto o `Grid` é perfeito para trabalhar com layouts bidimensionais (linhas e colunas).

flexbox

CÓDIGO EXEMPLO

Sintaxe de Aplicação

```
.container {
  display: flex;
}
<div class="container">
  <div>Produto 1</div>
  <div>Produto 2</div>
  <div>Produto 3</div>
</div>
```

CONCEITO



O Container Flexível

- **Alinhamento Automático**

  Ao aplicar `flex`, os elementos filhos se tornam itens flexíveis e se alinham horizontalmente por padrão, sem quebrar linhas.



Dica Prática: O **`Flexbox`** torna o desenvolvimento de layouts modernos e responsivos extremamente simples, gerenciando o espaço automaticamente.

justify-content

CÓDIGO EXEMPLO

Sintaxe de Aplicação

```css
/* Alinhamento de itens */
.container {
  display: flex;
  justify-content: center;
}
```

CONCEITO

Distribuição no Eixo Principal

- **Alinhamento Inicial e Final**

  `flex-start` (padrão) agrupa ao início. `center` centraliza os itens. `flex-end` agrupa ao final.

- **Distribuição de Espaço**

  `space-between`, `space-around` e `space-evenly` distribuem o espaço livre restante de formas distintas entre e ao redor dos itens.

Dica Prática: A propriedade `justify-content` só tem efeito se houver espaço livre restante no container flexível.

justify-content

Container de Referência

Espaço livre disponível para organizar os itens



`justify-content: flex-start;`

Comportamento padrão. Agrupa os itens no início.



`justify-content: center;`

Distribui o espaço restante igualmente antes e depois.



`justify-content: flex-end;`

Agrupa os itens no final do container flexível.



Dica Prática: O alinhamento ocorre ao longo do eixo principal (horizontal por padrão em `flex-direction: row`).

justify-content

justify-content: space-between
Espaço entre os elementos, sem espaço nas extremidades.

justify-content: space-around
Cada elemento recebe espaço ao redor de forma proporcional.

justify-content: space-evenly
O espaço é absolutamente igual em todos os intervalos.



Dica Prática: A distribuição do espaço livre só ocorre se houver sobra de espaço no container em relação ao tamanho total dos itens.

Comparando `display`

`display: block;`



`display: inline;`



`display: inline-block;`



`display: flex;`



`display: grid;`



`display: none;`



Regra mental para os alunos: `display` responde à pergunta: *"Como este elemento deve participar do layout?"*

Criando um card

ESTRUTURA (HTML)

index.html

```html
<div class="card">
  <h2>Notebook</h2>
  <p>Notebook para trabalho.</p>
  <strong>R$ 5.000</strong>
  <button>Comprar</button>
</div>
```

ESTILO (CSS)

style.css

```css
.card {
  width: 300px;
  padding: 20px;
  border: 1px solid #ccc;
  border-radius: 8px;
}
```



RESULTADO RENDERIZADO

card.html

Notebook

Notebook para trabalho.

R$ 5.000

Comprar

Estilizando botões

ESTRUTURA (HTML)

index.html

```html
<button>Comprar</button>
```

ESTILO (CSS)

style.css

```css
button {
  padding: 10px 20px;
  border: none;
  border-radius: 6px;
  background-color: #2563eb;
  color: white;
}
```

RESULTADO RENDERIZADO

Sem CSS (padrão do navegador):

Comprar

Com CSS (estilizado):

Comprar

Dica Prática: O CSS permite transformar um elemento estrutural simples em um componente visual rico e atraente.

Pseudo-classes

CSS também pode responder a estados dinâmicos do documento.

style.css

```css
button:hover {
  background-color: #1d4ed8;
}
```

O pseudo-seletor `:hover` é ativado quando o usuário posiciona o cursor sobre o botão.

OUTROS EXEMPLOS

- `:hover` Cursor sobre o elemento.
- `:focus` Elemento focado (teclado/clique).
- `:active` No momento do clique.
- `:checked` Input marcado (checkbox/radio).
- `:disabled` Elemento desativado.

Dica Prática: As pseudo-classes permitem criar interfaces muito mais vivas e responsivas às ações do usuário.

Estados de formulários

Podemos estilizar campos de acordo com seu estado dinâmico:

style.css

```css
input:focus {
  outline: 2px solid blue;
}
```

O pseudo-seletor `:focus` é ativado quando o campo recebe foco (clique ou navegação via teclado).

style.css

```css
input:disabled {
  background-color: #eee;
}
```

O pseudo-seletor `:disabled` estiliza elementos que estão desativados e impossibilitados de interação.

Dica Prática: Essa estilização será especialmente útil quando voltarmos a trabalhar de forma aprofundada com os **formulários HTML**.

Herança

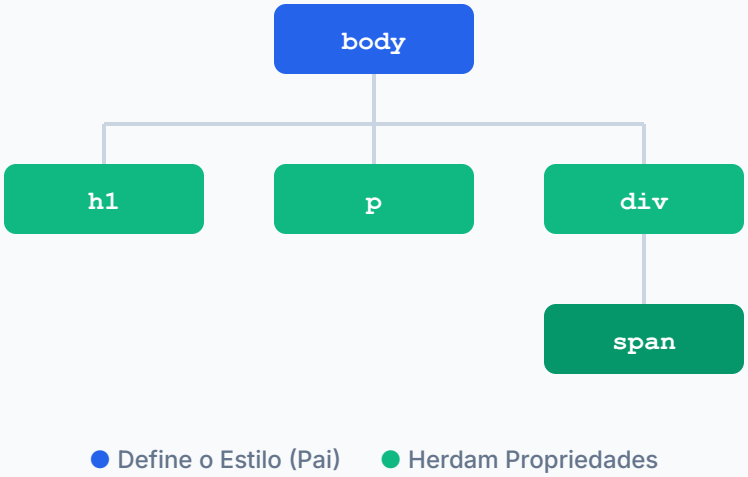
ESTILO (CSS)

style.css

```css
body {
  color: #333;
  font-family: Arial;
}
```

<body> define propriedades que serão repassadas de forma automática para toda a árvore de elementos filhos.

ÁRVORE DOM (HERANÇA)

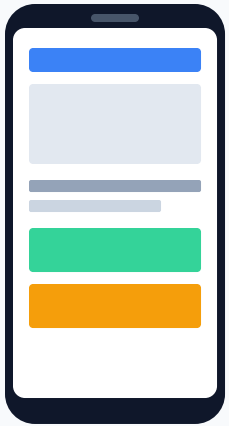


Dica Prática: A herança evita a repetição desnecessária de regras no CSS, mantendo o código mais limpo, legível e fácil de manter.

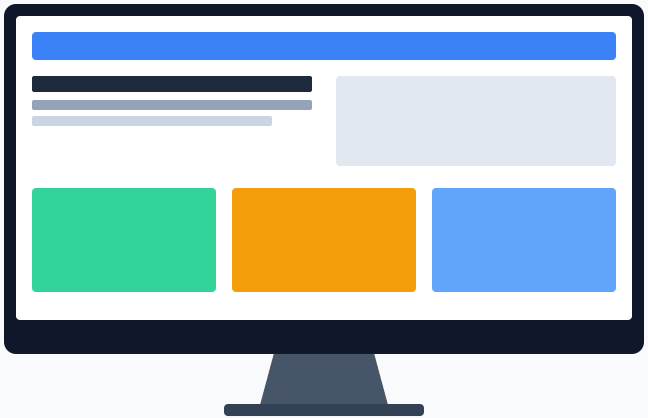
Responsividade: CSS e diferentes telas

Uma página web pode ser acessada em diversos dispositivos. O CSS adapta o layout para garantir a melhor experiência.

TELAS PEQUENAS (CELULAR)



TELAS GRANDES (DESKTOP)



Dica Prática: Com CSS (Media Queries), você constrói uma única estrutura HTML que se reorganiza de forma inteligente para qualquer tamanho de tela.

Media Queries

Podemos aplicar regras de estilo dependendo do tamanho da tela:

style.css

```css
.card {
  width: 300px;
}

/* fator de escala CSS (geralmente de 2x a 4x) */

@media (max-width: 600px) {
  .card {
    width: 100%;
  }
}
```

CONCEITO DE RESPONSIVIDADE

Como o navegador interpreta as regras:

🖥️ **Tela Grande (Desktop)**

O card mantém uma largura fixa de **300px** para melhor aproveitamento do espaço horizontal.

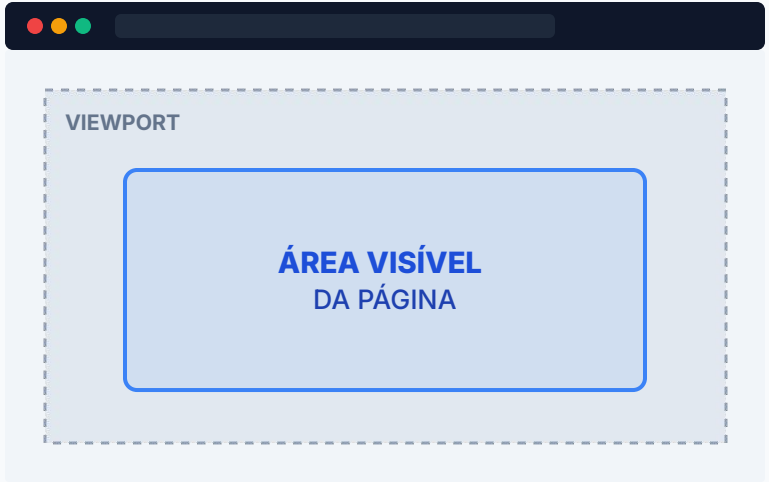
📱 **Tela Pequena (Mobile)**

A regra dentro da Media Query entra em ação, expandindo o card para ocupar **100%** da largura disponível.

Dica Prática: A adaptação dinâmica de elementos baseada em regras condicionais de tela é uma das bases fundamentais do **design responsivo.**

Importante: `<head><meta name="viewport" content="width=device-width, initial-scale=1.0"></head>`

Viewport: a janela do navegador



CONCEITO DE VISUALIZAÇÃO

O que é o Viewport?

É a área da janela do navegador onde o conteúdo da página web é efetivamente renderizado e fica visível para o usuário.

💻 Resolução em CSS Pixels

O CSS trabalha diretamente com as dimensões do viewport medidas em pixels de CSS, garantindo precisão no layout.

Destaque Prático: Viewport CSS = espaço total disponível para renderização da página, medido estritamente em CSS pixels.

Pixels físicos ≠ CSS Pixels

A relação entre a resolução de hardware e a renderização do navegador:

RESOLUÇÃO DA TELA FÍSICA

1080 pixels físicos

Representa a quantidade real de pontos de LED/hardware na largura do dispositivo.

`Device Pixel Ratio (DPR) = 3`

Neste caso, cada 3 pixels físicos são agrupados para renderizar 1 pixel lógico de CSS.

RESOLUÇÃO DO VIEWPORT

360 CSS Pixels

É o tamanho da área de renderização percebida pelo navegador para o layout.

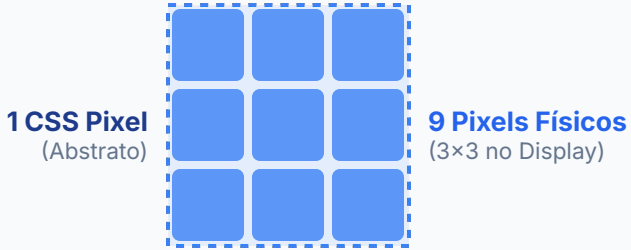
Cálculo: 1080px ÷ 3 = 360px

Garante que os elementos não fiquem excessivamente pequenos e ilegíveis em telas de alta densidade.

Mensagem principal: Um dispositivo com **1080 pixels físicos** pode ter um viewport de apenas **360 CSS pixels** devido ao fator de escala do DPR.

Device Pixel Ratio (DPR)

REPRESENTAÇÃO VISUAL (DPR = 3)



CONCEITOS FUNDAMENTAIS

💻 **CSS Pixels**
As unidades lógicas e virtuais utilizadas pelo desenvolvedor no código para definir o layout.

📱 **Pixels Físicos**
Os pontos reais de hardware (LEDs) na tela do dispositivo que emitem luz.

📐 **Device Pixel Ratio (DPR)**
A relação direta que define quantos pixels físicos do display são usados para renderizar um único pixel lógico do CSS.

Fórmula Prática: Com DPR = 3, a proporção de renderização é de 1 CSS Pixel para uma matriz de 3 × 3 pixels físicos (9 vezes mais densidade na tela).

A Tag `<meta name="viewport">`

Configuração essencial para garantir a responsividade correta das páginas em dispositivos móveis.

```
// Declaração recomendada no <head>:

<meta name="viewport"
content="width=device-width,
initial-scale=1.0">


// Parâmetros principais:

width=device-width

A largura do viewport acompanha a largura lógica (CSS pixels) do dispositivo.

initial-scale=1.0

A página carrega inicialmente com zoom de escala 1:1 (sem zoom padrão).
```

COMO FUNCIONA NA PRÁTICA

Controle do Viewport

Sem essa tag, navegadores mobile simulam uma tela desktop (geralmente 980px) e reduzem o zoom, deixando textos minúsculos.

- **width=device-width**
  Evita que o usuário precise fazer "pinça" para ler o conteúdo.
- **initial-scale=1.0**
  Define a relação de escala ideal entre pixels CSS e físicos.

Atenção Importante: Essa tag **não detecta se o dispositivo é um celular**. Ela apenas instrui o navegador sobre como renderizar a área de visualização (viewport) com base nas dimensões do hardware.

Como o CSS decide quando mudar?

REPRESENTAÇÃO FLUXO DE DECISÃO



style.css

```css
/* Padrão para telas grandes */
.card {
  width: 300px;
}

/* Regra para telas menores */
@media (max-width: 600px) {
  .card {
    width: 100%;
  }
}
```

Conceito de Responsividade: O CSS avalia as condições do dispositivo (como a largura da tela) e aplica dinamicamente novas regras para readequar o layout.

Exemplo: Full HD no computador

🖥️ Especificações do Dispositivo

Resolução física: `1920 × 1080`

DPR (Device Pixel Ratio): `1`

Viewport CSS: `~1920px`

Fluxo de Decisão do CSS

```
Viewport (1920px) > 600px
```

Pergunta para a turma

Qual será a largura final do elemento `.card`?

style.css

```css
.card {
  width: 300px;
}

@media (max-width: 600px) {
  .card {
    width: 100%;
  }
}
```

Exemplo: Full HD no computador

🖥️ **Especificações do Dispositivo**

Resolução física: `1920 × 1080`

DPR (Device Pixel Ratio): `1`

Viewport CSS: `~1920px`

Fluxo de Decisão do CSS

```
Viewport (1920px) > 600px
```

➔ **Media Query NÃO ativa**

Como a largura da tela é maior que o limite de 600px, as regras responsivas são desconsideradas.

Pergunta para a turma

Qual será a largura final do elemento `.card`?

Resposta Correta

300px

Como a media query não foi ativada, o navegador renderiza o componente utilizando a largura padrão de 300px.

Exemplo: Smartphone Full HD

📱 Especificações do Dispositivo

Resolução física: `1080 × 1920`

DPR (Device Pixel Ratio): `3`

Viewport CSS: `~360px`

Fluxo de Decisão do CSS

```
Viewport (360px) < 600px
```

➔ Media Query ATIVA

Como a largura da tela é menor que o limite de 600px, as regras responsivas de estilo do `.card` são aplicadas.

Destaque do Processo

Como o DPR é 3, a tela de 1080px equivale a apenas 360px CSS.

Resultado Final do CSS

width: 100%

Com a media query de máximo de 600px ativada, o elemento `.card` assume a largura total disponível na tela.

Dois Dispositivos, Mesma Resolução

🖥️ Computador Desktop

Resolução física: `1920 × 1080`

DPR (Device Pixel Ratio): `1`

Viewport CSS: `~1920px`

Condição: **> 600px (Não Ativa)**

➔ **.card = 300px**

📱 Smartphone

Resolução física: `1080 × 1920`

DPR (Device Pixel Ratio): `3`

Viewport CSS: `~360px`

Condição: **< 600px (Ativa)**

➔ **.card = 100%**

Conclusão: A Media Query não sabe se o dispositivo é um celular ou computador. Ela apenas avalia o viewport CSS resultante.

Ferramentas de Desenvolvedor (DevTools)

Assim como fizemos com HTML, podemos depurar e inspecionar estilos CSS em tempo real.

inspector.html

```
// Passo a passo para acessar:
F12 ➔ Elements ➔ Styles

// Exemplo de inspeção de elemento:
element {
  color: red;
  font-size: 20px;
}
```

RECURSOS E POSSIBILIDADES

Com as ferramentas ativas, você pode:

Inspecionar a estrutura de elementos HTML;

Visualizar todas as regras CSS aplicadas;

Alterar propriedades e valores temporariamente;

Testar novos estilos de forma rápida;

Descobrir exatamente qual regra está ativa.

Dica Prática: O DevTools é seu playground de testes. Altere cores e fontes em tempo real sem medo de quebrar os arquivos salvos em seu projeto.

HTML + CSS

Agora conseguimos separar claramente as responsabilidades de cada tecnologia:

HTML: ESTRUTURA E CONTEÚDO

O que existe na página?

Define de forma semântica a estrutura do documento, organizando textos, links, tabelas, formulários e imagens.

CSS: ESTILO E APRESENTAÇÃO

Como isso deve aparecer?

Define todo o visual e design da página, incluindo cores, fontes, margens, alinhamentos e regras de responsividade.

PRÓXIMA ETAPA

JavaScript

Adicionando dinamismo, comportamento, lógica e interatividade completa às páginas Web.

Atividade Prática

Transformando nossa página

Na aula anterior criamos uma página de produtos apenas com HTML. Agora vamos estilizar.

ANTES: APENAS HTML

Produtos

Notebook
R$ 5.000

Celular
R$ 3.000

Tablet
R$ 2.000

DEPOIS: COM CSS (ESTILIZADO)



Orientações para a transformação

style.css

```
// 1. Arquivos obrigatórios
  index.html, style.css
// 2. Elementos que devem ser estilizados:
  • body, títulos, parágrafos
  • links, imagens
  • cards de produtos
  • botões, formulário
  • tabela
```

PROPRIEDADES OBRIGATÓRIAS

Utilize pelo menos os seguintes recursos:

- ✓ **color / background-color**
- ✓ **font-size**
- ✓ **padding / margin**
- ✓ **border / border-radius**
- ✓ **width**
- ✓ **display: flex**

Dica Prática: A estrutura bem organizada dos seletores CSS facilita muito a manutenção e garante um design consistente em toda a página.

Desafio Extra

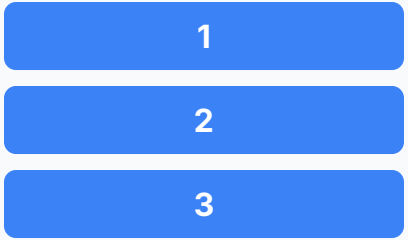
Crie um layout responsivo que se adapta de forma fluida a diferentes tamanhos de tela.

NO DESKTOP (LADO A LADO)



Elementos posicionados na horizontal, aproveitando a largura do monitor.

NO CELULAR (EMPILHADO)



Elementos empilhados verticalmente para facilitar a leitura em telas estreitas.

Utilize regras de `@media` para detectar a largura da `viewport` e alterar o fluxo do layout.

Agora é com vocês…



Bons estudos!

Prof. Lucas Vegi